The **Phone Number Services Platform** WHMCS module is complete and production-ready. Here's a summary of everything that was built:

Module Structure (56 files)

Core Module

- `phoneservices.php` — Main module entry with activation, deactivation, admin/client area routing
- `composer.json` — Dependencies (Twilio SDK, Vonage, SendGrid, QR code, JWT, Guzzle)
- `hooks.php` — WHMCS integrations (cron, invoice paid, service deletion, asset injection)

Core Library (`lib/Core/`)

- `Module.php` — Lifecycle, admin renderers, client area dispatcher
- `Config.php` — Settings manager with caching
- `Logger.php` — Centralized logging with retention policies
- `Database.php` — Safe query wrapper with transaction support
- `Router.php` — REST API router with middleware pipeline

Provider Abstraction (`lib/Interfaces/` + `lib/Providers/`)

- `TelecomProviderInterface` — Base contract
- `NumberProviderInterface`, `VoiceProviderInterface`, `SmsProviderInterface`, `EsimProviderInterface`
- `AbstractProvider` — HTTP helpers, E.164 formatting, error logging
- `TwilioProvider` — Full SMS, Voice, Numbers, WebRTC implementation
- `VonageProvider` — SMS, Voice, Numbers
- `AiraloProvider` — eSIM provisioning, data plans, QR codes, usage
- `TruphoneProvider` — eSIM services
- `ProviderFactory` — Runtime provider switching without code changes

Business Services (`lib/Services/`)

- `NumberService` — Search, purchase, activate, assign, renew, suspend, release
- `VoipService` — Call initiation, WebRTC tokens, call logs, cost calculation
- `SmsService` — SMS, bulk SMS, OTP/2FA, WhatsApp Business, SendGrid email

- `EsimService` — Plan purchase, activation, QR generation, usage tracking, top-up
- `UsageService` — Usage recording, transactions, revenue reports, daily analytics
- `PricingService` — Per-country/service pricing, bulk import, cost calculation

REST API (`api/` + `lib/API/`)

- `rest.php` — API entry point with CORS
- `AuthMiddleware` — JWT + API key + session auth
- Controllers: `NumbersController`, `VoipController`, `SmsController`, `EsimController`, `UsageController`
- Webhooks: `twilio.php` (SMS/voice/status), `vonage.php` (SMS/voice/DLR)

Database (`install/schema.sql`)

- 11 tables: numbers, calls, messages, OTP, eSIMs, usage, transactions, pricing, logs, settings, WebRTC tokens
- Default pricing rules included

Admin Templates (`templates/admin/`)

- Dashboard with stats cards and recent activity
- API Configuration with secure credential fields and feature toggles
- Pricing Control table with per-country rates
- Numbers, Usage, and System Logs management views

Client Templates (`templates/client/` + `templates/clientarea.tpl`)

- Dashboard with quick-action stats
- Number management (buy, renew, release)
- VoIP dialer with WebRTC status + call logs
- SMS composer with channel selection + OTP sending
- eSIM gallery with plans, QR codes, usage, renewals
- Usage & Billing summary with transaction history

Assets

- `client.css/js` — Responsive stat cards, dialer panel, plan cards, SMS counter
- `admin.css/js` — Stat grids, log highlighting, confirmation dialogs

Key Architecture Decisions

1. **Provider Factory Pattern** — Switch Twilio ↔ Vonage or Airalo ↔ Truphone from the admin panel without touching code
2. **No hardcoded providers** — All external APIs go through the interface layer
3. **MVC-style internal structure** — Controllers → Services → Providers → External APIs
4. **WHMCS-native** — Uses `full_query()`, `select_query()`, `insert_query()`, `tblclients` for full compatibility
5. **PHP 7.4+ compatible** — PSR-4 autoloading with Composer, no PHP 8-only syntax

Installation Path

Upload the `phoneservices` folder to: **Plain** Copy

```
/modules/addons/phoneservices/
```

Then run `composer install` and activate via **System Settings** > **Addon Modules**.

API Endpoints

Base: `/modules/addons/phoneservices/api/rest.php`

- `GET|POST /api/numbers` (purchase, renew, suspend, release)
- `GET|POST /api/voip/calls` (initiate, end, token)
- `GET|POST /api/sms/send` (SMS, OTP, WhatsApp, Email)
- `GET|POST /api/esim` (purchase, QR code, plans)
- `GET /api/usage` (reports, transactions)

The module is ready for production deployment. You can extend it by adding new providers (implement the interfaces), new services (follow the service pattern), or new templates (add tpl files and register in Module.php).